# An Overview of Convolutional Neural Networks and Recurrent Neural Networks

## Motivation

Different kinds of data possess different kinds of structure.

- Images possess spatial structure.
- Text, speech and time-series data possess temporal structure.

CNNs are designed to exploit spatial structure, while RNNs are designed to exploit temporal structure.

## Convolutional Neural Networks (CNNs)

### Key Idea

A small filter (kernel) examines a local region of the image, and the same kernel is reused across the entire image. This allows the network to detect local spatial patterns while preserving spatial relationships between nearby pixels.

### Convolution and Feature Maps

Given an image $I$ and a kernel $K$,

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n)K(m, n).$$

In practice, CNN implementations use the closely related operation of cross-correlation.

At each location, the kernel and the corresponding image patch are viewed as vectors, and their dot product is computed. As the kernel slides across the image, these dot products form a **feature map**.

A typical CNN pipeline is

$$\text{Image} \to \text{Convolution} \to \text{Activation} \to \text{Pooling} \to \text{Flatten} \to \text{Dense Layers (NN)} \to \text{Output}.$$

### Example

Consider the images $I_1 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$ and $I_0 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$.

Suppose the kernel is $K = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$. (This kernel detects vertical strokes.)

For the first window of $I_1$, that is,

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix},$$

we obtain

$$0 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = 1.$$

Moving the kernel one step to the right gives output 3.
Moving it another step gives output 0.
Thus the first row of the feature map is $[1 \quad 3 \quad 0]$.
After cross-correlation (dot products at all locations), we get

$$I_1 * K = \begin{bmatrix} 1 & 3 & 0 \\ 1 & 3 & 0 \\ 1 & 3 & 1 \end{bmatrix}$$

and

$$I_0 * K = \begin{bmatrix} 2 & 1 & 2 \\ 3 & 0 & 3 \\ 2 & 1 & 2 \end{bmatrix}.$$

Then a bias term is added to each entry. Suppose the bias term is -2. After adding the bias term, we get the feature maps as follows:

$$F_1 = \begin{bmatrix} -1 & 1 & -2 \\ -1 & 1 & -2 \\ -1 & 1 & -1 \end{bmatrix}$$

and

$$F_0 = \begin{bmatrix} 0 & -1 & 0 \\ 1 & -2 & 1 \\ 0 & -1 & 0 \end{bmatrix}.$$

Note that the middle column of $F_1$ contains consistently positive values, indicating the presence of a strong vertical stroke.

**Activation Function**

A nonlinear activation function is applied on the feature maps.
The most common choice is ReLU:

$$\text{ReLU}(x) = \max\{0, x\}.$$

We get

$$\text{ReLU}(F_1) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

and

$$\text{ReLU}(F_0) = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

**Pooling**

Pooling reduces the size of the feature map while retaining important information; and hence reduces memory usage and computational cost.

In this example, applying max pooling (taking maximum in each $2 \times 2$ submatrix) to $\text{ReLU}(F_1)$ gives

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Applying max pooling to $\text{ReLU}(F_0)$ gives

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Note that the flattened feature vectors are $[1, 1, 1, 1]$ and $[1, 1, 1, 1]$, which are indistinguishable. For this very small toy example, pooling discards too much information and makes the feature vectors indistinguishable.

However, pooling is optional. So for this example, we can flatten $\text{ReLU}(F_1)$ and $\text{ReLU}(F_0)$ directly.

After flattening, we get the feature vectors as

$$[0, 1, 0, 0, 1, 0, 0, 1, 0]$$

and

$$[0, 0, 0, 1, 0, 1, 0, 0, 0]$$

respectively.

Alternatively, we can apply average (mean) pooling. With average pooling, $\text{ReLU}(F_1)$ gives

$$\begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}.$$

and $\text{ReLU}(F_0)$ gives

$$\begin{bmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{bmatrix}.$$

After flattening, we get the feature vectors as

$$[0.5, 0.5, 0.5, 0.5]$$

and

$$[0.25, 0.25, 0.25, 0.25],$$

which are distinguishable although the orientation information has still been lost.

In practical CNNs, pooling is generally applied to larger feature maps and across many feature channels.

**Dense Layers**

The flattened feature vector is fed into one or more fully connected neural-network layers.

Thus:

- Convolutional layers act as feature extractors.
- Dense layers act as classifiers.

# Recurrent Neural Networks (RNNs)

A standard feedforward neural network processes each input independently and does not maintain a memory of previously processed inputs. However, for sequential data such as text, the order of the inputs is important. For example,

> Dog bites man.

and

> Man bites dog.

contain the same words but have different meanings because the words occur in different orders. Recurrent neural networks (RNNs) are designed to process sequential data. They maintain a **hidden state** that carries information from earlier time steps to later time steps. At each time step $t$, the recurrent hidden layer produces a hidden-state vector $h_t$. This state depends on the current input $x_t$ and the previous hidden state $h_{t-1}$, allowing information to be carried through the sequence.

## Hidden State

The central RNN equation is

$$h_t = f\left(W_h h_{t-1} + W_x x_t + b\right),$$

where $x_t$ is the input at time step $t$, $h_{t-1}$ is the previous hidden state, $W_x$ and $W_h$ are weight matrices, $b$ is a bias vector, $f$ is a nonlinear activation function (commonly tanh) and $h_t$ is the updated hidden state. The hidden state $h_t$ acts as a memory that summarizes the information processed by the RNN up to time step $t$.

To illustrate the idea of memory, consider a highly simplified RNN. We make the following assumptions:

- The input $x_t$ and hidden state $h_t$ are scalars;
- The recurrent weight and input weight are both equal to 1;
- The bias is 0;
- The activation function is the identity function, $f(z) = z$.
- The initial hidden state is set to zero, $h_0 = 0$.

Under these assumptions, the general RNN equation

$$h_t = f\left(W_h h_{t-1} + W_x x_t + b\right)$$

reduces to

$$h_t = h_{t-1} + x_t, \qquad h_0 = 0.$$

Let

$$x_1 = 2, \qquad x_2 = 3, \qquad x_3 = 4.$$

Then

$$\begin{aligned} h_1 &= h_0 + x_1 = 0 + 2 = 2, \\ h_2 &= h_1 + x_2 = 2 + 3 = 5, \\ h_3 &= h_2 + x_3 = 5 + 4 = 9. \end{aligned}$$

Thus,

$$h_3 = x_1 + x_2 + x_3 = 2 + 3 + 4 = 9.$$

In this simplified example, the hidden state stores the cumulative sum of all the inputs received up to the current time step. It therefore illustrates how an RNN can carry information forward through a sequence.

### Vanishing and Exploding Gradients

To illustrate the source of the vanishing and exploding gradient problems, consider a simplified RNN:

$$h_t = f(w_h h_{t-1} + w_x x_t).$$

For simplicity, assume that

- the inputs and hidden states are scalars;
- the activation function is the identity function, $f(z) = z$;
- the bias is zero.

The recurrence then becomes

$$h_t = w_h h_{t-1} + w_x x_t.$$

Expanding the recurrence gives

$$h_1 = w_h h_0 + w_x x_1,$$

$$\begin{aligned} h_2 &= w_h h_1 + w_x x_2 \\ &= w_h(w_h h_0 + w_x x_1) + w_x x_2 \\ &= w_h^2 h_0 + w_h w_x x_1 + w_x x_2, \end{aligned}$$

and

$$\begin{aligned} h_3 &= w_h h_2 + w_x x_3 \\ &= w_h^3 h_0 + w_h^2 w_x x_1 + w_h w_x x_2 + w_x x_3. \end{aligned}$$

More generally,

$$h_t = w_h^t h_0 + \sum_{k=1}^{t} w_h^{t-k} w_x x_k.$$

Thus, information from an earlier time step is repeatedly multiplied by the recurrent weight $w_h$. For example, the contribution of $x_1$ to $h_t$ contains the factor

$$w_h^{t-1}.$$

During training, an RNN is unfolded through time, and gradients are propagated backward through these repeated recurrent computations. Consequently, the derivatives also contain products involving the recurrent weights and the derivatives of the activation function. Therefore:

- if the magnitudes of these repeated factors are less than 1, their products can approach zero, leading to the **vanishing gradient problem**;
- if their magnitudes are greater than 1, their products can grow rapidly, leading to the **exploding gradient problem**.

Vanishing gradients make it difficult for the RNN to learn long-range dependencies because information from much earlier time steps has very little influence on parameter updates. Exploding gradients can cause excessively large parameter updates and unstable training.

Applications of RNNs include language modelling, machine translation, speech recognition, and time-series forecasting. The vanishing and exploding gradient problems encountered in basic RNNs motivated the development of more advanced recurrent architectures, such as Long Short-Term Memory networks (LSTMs) and Gated Recurrent Units (GRUs), which are designed to learn long-term dependencies more effectively.